

AI-BASED INTELLIGENT CROWD MONITORING AND REAL-TIME EVACUATION ASSISTANCE SYSTEM

S. KRISH PRIYAN – 24623127

SANJAY S – 24623146

IMMANUEL R – 24623136

BATCH NO. 14

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

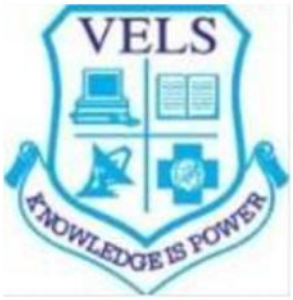
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)



VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**AI-BASED INTELLIGENT CROWD MONITORING AND REAL-TIME EVACUATION ASSISTANCE SYSTEM**" is the bonafide work of **S. KRISH PRIYAN – 24623127**, **SANJAY S – 24623146** and **IMMANUEL R – 24623136**, who carried out the project under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. ULAGA PRIYA, M.E., Ph.D.
Associate Professor
Head of the Department
Department of AIML
School of Engineering
VISTAS, Chennai

SIGNATURE OF THE SUPERVISOR
Ms. B. REKHA DEVI, M.Tech
Assistant Professor
Department of AIML
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva-Voce examination held on 30 October 2026 at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the mini project report entitled "AI-BASED INTELLIGENT CROWD MONITORING AND REAL-TIME EVACUATION ASSISTANCE SYSTEM", submitted to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence), is a record of original work carried out by us under the guidance of Ms. B. Rekha Devi,M.Tech, Assistant Professor, Department of Artificial Intelligence and Machine Learning. The work has not been submitted, in part or in full, for the award of any other degree or diploma.

Place: Chennai

Date: 30 October 2026

S. KRISH PRIYAN – 24623127

SANJAY S – 24623146

IMMANUEL R – 24623136

ACKNOWLEDGEMENT

It is a great pleasure and privilege to acknowledge the help and support we received in completing this project. The infrastructure and laboratory facilities provided by our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH**, have been a constant source of motivation, and we take this opportunity to thank him. We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya, M.E., Ph.D.**, for continuous support and encouragement throughout the project. We extend our sincere thanks to our **PROJECT GUIDE, Ms. B. REKHA DEVI, Assistant Professor, Department of Artificial Intelligence and Machine Learning**, for her guidance throughout this project, and to the Department of Computer Science and Engineering, VISTAS, for the facilities provided. We also thank the Project Coordinator Ms. F.Benasir Begam ME.,(Ph.D), faculty members, librarian and non-teaching staff for their help, and our parents and friends for their support and encouragement.

S. KRISH PRIYAN – 24623127

SANJAY S – 24623146

IMMANUEL R – 24623136

ABSTRACT

Large public gatherings such as festivals, concerts, railway stations, shopping malls, and sports events often face challenges related to overcrowding, panic, and poor crowd management, which can lead to accidents or stampedes. The proposed AI-Based Intelligent Crowd Monitoring and Emergency Evacuation Assistance System uses Artificial Intelligence, Computer Vision, and real-time mapping to monitor crowd movement continuously through surveillance cameras. The system analyses crowd density and movement patterns to detect abnormal behaviour and predict potential risks before they become critical. When a dangerous situation is identified, it immediately alerts the concerned authorities and generates a live crowd map by dividing the area into multiple regions based on crowd density. It then recommends the safest evacuation routes and provides a region-wise movement sequence to ensure an organised evacuation, reducing congestion and preventing bottlenecks. The application also offers a real-time visual dashboard that helps authorities monitor crowd conditions and make faster, data-driven decisions during emergencies. By integrating intelligent monitoring, predictive analysis, automated alerts, and dynamic evacuation planning into a single platform, the proposed system aims to improve public safety, minimise response time, and significantly reduce the risk of crowd-related disasters in large public spaces.

Keywords: Real-Time Surveillance, Abnormal Behaviour Detection, Emergency Alert System, Smart Evacuation, Crowd Monitoring, Computer Vision, Path Planning, Region-Based Crowd Segmentation.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES AND TABLES	iii
	LIST OF ABBREVIATIONS	iv
1	**CHAPTER 1 – INTRODUCTION**	1
	1.1 Overview of the project	1
	1.2 Scope and objective	2
2	**CHAPTER 2 – REVIEW OF THE LITERATURE**	4
	2.1 Literature overview	4
	2.2 Research summary	4
3	**CHAPTER 3 – SYSTEM DESIGN**	8
	3.1 Existing system	8
	3.2 Proposed system	8
	3.3 System architecture	9
	3.4 Data flow / workflow	10
	3.5 Hardware requirements	12
	3.6 Software requirements	13
4	**CHAPTER 4 – IMPLEMENTATION**	14
	4.1 Methodology	14
	4.2 Implementation	14
	4.2.1 Video Input and Pre-processing Module	14
	4.2.2 Person Detection and Tracking Module	14
	4.2.3 Zone Load and Forecast Module	15
	4.2.4 Risk Classification and Alert Module	15
	4.2.5 Fire Detection Module	16
	4.2.6 Predictive Bottleneck Solver	16
	4.2.7 Downstream Risk and Bottleneck Alert Module	17
	4.2.8 Dashboard and Alert Module	17
	4.3 Module integration	17
5	**CHAPTER 5 – RESULTS, CONCLUSION AND FUTURE WORK**	18
	5.1 Results and evaluation	18
	5.2 Discussion	20
	5.3 Challenges and limitations	21
	5.4 Conclusion	21
	5.5 Future work	21
	REFERENCES	23

CHAPTER NO.	TITLE	PAGE NO.
	APPENDIX – SOURCE CODE	25

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Use-case diagram	9
3.2	Architecture of the crowd monitoring and evacuation assistance system	10
3.3	Data-flow diagram – Level 0 (context)	11
3.4	Data-flow diagram – Level 1	11
3.5	Per-frame processing workflow	12
4.1	Project Gantt chart (14 weeks)	14
4.2	Venue zone-flow graph used by the bottleneck solver (AEGIS-CMS)	17
5.1	Dashboard screenshot from the project presentation	18
5.2	Final dashboard with the bottleneck-solver table	19
5.3	Detection output and threat-monitor alerts	19

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of existing approaches with the proposed system	5
2.2	Base papers	6
3.1	Functional requirements	9
3.2	Modules and their responsibilities	10
3.3	Hardware requirements	12
3.4	Software requirements	13
4.1	Risk levels and system response	16
5.1	Observed prototype behavior (from the conference paper)	18
5.2	Test cases	20

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
AR	Augmented Reality
CCTV	Closed-Circuit Television
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CSS	Cascading Style Sheets
CUDA	Compute Unified Device Architecture
DFD	Data-Flow Diagram
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
mAP	mean Average Precision
PPO	Proximal Policy Optimization
RAM	Random Access Memory
SSD	Solid-State Drive
UAV	Unmanned Aerial Vehicle
YOLO	You Only Look Once

CHAPTER 1 – INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

Large public gatherings such as festivals, concerts, railway stations, shopping malls, sports events and college events can experience overcrowding, panic and uncontrolled crowd movement, which can lead to accidents or stampedes. Crowd disasters rarely start with one dramatic event: they build quietly, as a corridor fills faster than it drains, until a small push becomes a surge. Helbing et al. showed in simulation that when hurried people compete for the same exit, the crowd can arch and clog around it, so the escape ends up slower than a calm one [1].

Most venues still rely on people to catch the moment when a zone becomes dangerous. An operator watching a wall of CCTV feeds has to spot a filling corridor, judge whether it is dangerous, decide where people should go and tell someone, all while the situation keeps changing. Cameras record what happens, but they do not count, compare or warn. The **AI-Based Intelligent Crowd Monitoring and Real-Time Evacuation Assistance System** addresses this gap. It treats each camera feed as a sensor, turns it into per-zone crowd figures, and passes those figures straight into an evacuation planner.

Problem Statement

Traditional CCTV set-ups provide raw video feeds but lack automated, continuous analysis for:

- Live spatial crowd counting and density estimation.
- Identification of high-density risk zones before they become critical.
- Real-time tracking of changing conditions during an evacuation.
- Determining safe exit routes, which is difficult and slow to do manually during a rapidly developing emergency.

The problem addressed by this project is to replace manual video inspection with an automated AI system capable of real-time crowd-density estimation, risk detection, visual zone mapping, dynamic alerts and congestion-aware evacuation route recommendations.

System Overview

The proposed system is an end-to-end AI-based decision-support prototype organized in four layers:

- **AI surveillance layer:** YOLOv8-based person detection and real-time tracking from video feeds.
- **Spatial analysis layer:** segmentation of the venue into zones with automated load calculation and a short-horizon forecast.
- **Risk and alert layer:** automated risk classification (safe, moderate, critical) and fire detection, triggering immediate alerts.
- **Live mapping and evacuation layer:** a live crowd map and congestion-aware path finding (A*/Dijkstra) with a region-wise movement sequence, presented on one web dashboard.

Advantages of the System

- **Proactive decision support:** crowd management moves from passive CCTV watching to automated counting, forecasting and alerting.
- **Routes that follow the crowd:** evacuation guidance avoids corridors that are already filling, rather than always choosing the shortest path on a map.
- **Organized evacuation:** a region-wise release order reduces congestion and bottlenecks at exits.
- **Single platform:** live video, capacity tables, forecasts, flow, fire status and the event log appear on one dashboard.
- **Uses existing cameras:** ordinary CCTV feeds or webcams are sufficient; evacuees need no devices.

Applications

- **Public events:** festivals, concerts and sports grounds, where crowds gather and disperse quickly.
- **Transport hubs:** railway platforms and stations at peak hours.
- **Commercial spaces:** shopping malls and food courts.
- **Campuses and smart cities:** college events and city-scale surveillance networks, including monitoring crowd counts for distancing.

1.2 SCOPE AND OBJECTIVE

Objectives

1. To detect and track people in multi-camera video using YOLOv8.
2. To divide the venue into zones and report each zone's load against its allotted capacity.
3. To classify each zone as safe, moderate or critical and forecast when a zone will reach capacity.
4. To detect fire in the camera feeds and treat the affected zone as blocked.

5. To compute congestion-aware evacuation routes with A*/Dijkstra and a region-wise movement sequence.
6. To present live panels, tables, alerts and an event log on a real-time web dashboard for authorities.

Scope of the Project

- **Venues:** festivals, concerts, railway stations, shopping malls, sports grounds and college events.
- **Input:** CCTV streams, webcams or uploaded video clips; the prototype console handles four channels.
- **Technology:** Python, OpenCV, PyTorch, Ultralytics YOLOv8, NumPy, Flask, HTML5, CSS3, JavaScript and SQLite (PostgreSQL optional).
- **Outside the current scope:** large-scale quantitative evaluation, density-map counting for very dense scenes, push alerts, digital signage and edge deployment. These are future work (Section 5.5).

CHAPTER 2 – REVIEW OF THE LITERATURE

2.1 LITERATURE OVERVIEW

Research has produced good tools for each part of the crowd-safety problem: networks that count heads, trackers that follow individuals, behavior models that flag anomalies, and path-planning methods that find exits. What is rarely found is a working system in which the live count of a zone changes the route recommended for that zone. This chapter reviews crowd counting, detection and tracking, behavior and anomaly detection, evacuation modeling and recent integrated systems, and identifies the gap that this project addresses.

2.2 RESEARCH SUMMARY

Crowd Counting and Density Estimation

In dense crowds people occlude one another and detection-based counting struggles, so much of the literature regresses a density map instead. Zhang et al. introduced a multi-column CNN that copes with the changing head size caused by perspective [2]. CSRNet used a VGG-16 front end with dilated convolutions to keep spatial resolution and reported strong results on congested scenes, including the ShanghaiTech dataset [3]. Sindagi and Patel survey these CNN-based methods and their weak points, such as illumination changes and the cost of annotation [4]. These models tell us how many people are present, but they do not say where people should go next.

Person Detection and Tracking

The YOLO family reframed detection as a single forward pass, which makes real-time video practical [5]. Current releases such as YOLOv8 offer small variants that run on a modest GPU [6], and the COCO-trained weights include a reliable person class [7]. DeepSORT couples a motion model with an appearance descriptor to keep identities stable through short occlusions [8]. Khel et al. used a hybridized, pruned YOLOv4 with an attention module to count people and estimate their speed and direction in real time, reporting 92.1% mAP on the JHU dataset with lower memory use [9]. DeepSOCIAL combined YOLOv4, SORT tracking and inverse perspective mapping to monitor social distancing, reporting 99.8% mAP at 24.1 fps on the Oxford Town Centre dataset [10]. These systems monitor crowds but do not define risk zones or evacuation routes.

Crowd Behavior and Anomaly Detection

Social LSTM predicts pedestrian paths by letting neighboring trajectories influence each other [11], which is useful for navigation but heavy for dense real-time scenes. Mahadevan et al.

modeled normal appearance and motion with mixtures of dynamic textures and flagged deviations [12]. Direkoğlu fed optical-flow motion information images to a CNN to detect panic and escape behavior on the UMN and PETS2009 datasets [13]; dense optical flow [14] remains a cheap way of spotting sudden dispersal. These methods raise an alarm, but the alarm is not linked to an evacuation plan.

Evacuation Modeling and Path Planning

Evacuation research has mostly concentrated on simulation. Social-force models explain how panic produces jams at doors [1], and Dijkstra's algorithm and A* remain the standard graph-search tools [15], [16]. In most studies the graph is fixed and edge weights are physical distances, so a corridor that is shortest on paper but packed with people is treated like an empty one. Pedestrian planning guidance grades walkways by the space available per person [17]. Zhang et al. built a population density map from evacuees' augmented-reality devices and used a modified A* to balance travel time against congestion [18], but this requires every evacuee to carry an AR device. Huang et al. combined YOLOv3 pedestrian detection and positioning with a cellular-automata evacuation simulation [19], which depends on a separate simulation set-up.

Recent Integrated Systems

Drishti AI-Event Guardian merges CCTV and drone footage on a cloud platform and uses YOLOv8 for density estimation, anomaly detection and crowd-flow forecasting, together with guard reallocation [20]; it is cloud- and UAV-heavy and gives no route guidance to the crowd. GPEvac combines a graph neural network with Proximal Policy Optimization to compute evacuation routes during shooting events [21]; it requires reinforcement-learning training and targets a specific threat scenario. Both are recent arXiv preprints.

Table 2.1: Comparison of existing approaches with the proposed system

Approach	What it provides	Live video	Routes adapt to live crowd	Alerts / dashboard
Density-map counting [2], [3]	Head count and density map from an image	Frame by frame	No	No
YOLO detection with DeepSORT [5], [8]	Detects and follows individuals	Yes	No	No
Real-time monitoring [9], [10]	Count, speed, direction or distancing risk	Yes	No	Monitoring only
Social LSTM [11]	Predicts pedestrian trajectories	Needs tracks	No	No
Anomaly detection [12], [13]	Flags abnormal motion	Yes	No	Alarm only

Approach	What it provides	Live video	Routes adapt to live crowd	Alerts / dashboard
Escape-panic simulation [1]	Explains jamming at exits	No (offline)	No	No
Dijkstra / A* routing [15], [16]	Shortest path on a graph	No (static graph)	Only if costs are updated	No
AR congestion-aware routing [18]	Density map from AR devices; modified A*	Device positions	Yes	No
Proposed system	Counts, classifies risk, forecasts, routes and alerts	Yes (multi-channel)	Yes	Yes

Base Papers

Table 2.2 summarizes the five base papers identified for the project, with the goal, method, dataset, reported result and limitation of each, as presented in the project review.

Table 2.2: Base papers

Paper	Method and dataset	Reported result	Limitation for this project
CSRNet, Li et al., 2018 [3]	VGG-16 with dilated convolutions; ShanghaiTech Parts A and B	MAE 68.2 on Part A	Needs well-annotated data; affected by lighting changes
Social LSTM, Alahi et al., 2016 [11]	LSTM with a social pooling layer; ETH and UCY	More accurate path prediction than earlier methods	Heavy for real-time dense crowds; short look-ahead
Direkoğlu, 2020 [13]	Optical-flow motion images fed to a CNN; UMN and PETS2009	Detects panic and escape behavior	Detects events only; no exit guidance
Khel et al., 2023 [9]	Hybridized, lighter YOLOv4; JHU dataset	Lower memory use; close to real-time	Monitoring only; no risk zones or routes
DeepSOCIAL, Rezaei and Azarmi, 2020 [10]	YOLOv4, SORT tracking, inverse perspective mapping; Oxford Town Centre	99.8% mAP at 24.1 fps	Built for distancing risk, not evacuation

Research Gap

- Recent YOLO systems count, measure density or raise alerts, but do not guide evacuation.
- Evacuation routing mostly uses simulation, reinforcement learning or user devices such as AR headsets, not ordinary CCTV.
- Congestion-aware routing exists but is rarely combined with live detection and an operator dashboard.
- Large platforms need cloud infrastructure, drones and many cameras.

This project positions itself as a lightweight, CCTV-only system: it detects people, measures zone load, classifies risk, routes around congestion with A*/Dijkstra, and shows everything on a live dashboard.

CHAPTER 3 – SYSTEM DESIGN

3.1 EXISTING SYSTEM

In existing venues, crowd safety depends on operators watching CCTV screens, on stewards on the ground and on fixed evacuation plans drawn up in advance. The cameras record but do not analyze, so a corridor that is filling is noticed only when an operator happens to look at it. Evacuation plans assign each area to the nearest exit regardless of how crowded the route is on the day, and everyone is typically released at once.

Limitations of the Existing System

- No automated counting or density estimation; risk is judged by eye.
- Late detection: a zone may be noticed only after it has become dangerous.
- Static exit plans ignore live congestion, so people are sent into corridors that are already full.
- Simultaneous release causes queues and bottlenecks at shared exits.
- Information is scattered across screens, radios and paper plans.

3.2 PROPOSED SYSTEM

The proposed system converts live surveillance feeds into actionable crowd intelligence and evacuation guidance. It counts people in every zone, labels each zone safe, moderate or critical, forecasts how soon a zone will reach its allotted capacity, checks for fire, and, when a zone turns critical or a fire is confirmed, raises an alert and computes congestion-aware evacuation routes with a staggered region-wise release order. Figure 3.1 shows the users and their use cases.

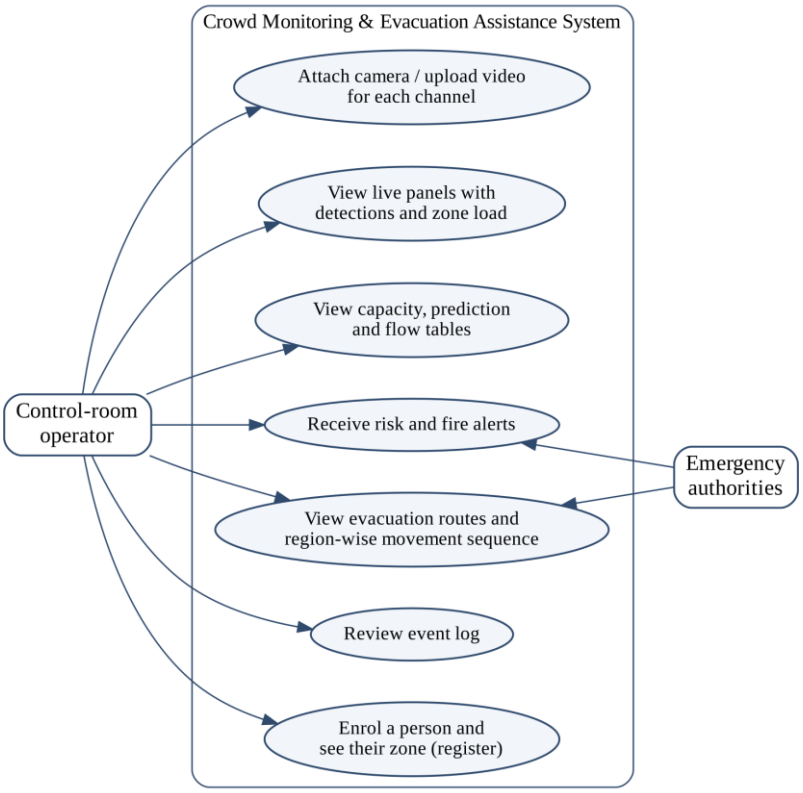


Figure 3.1: Use-case diagram

Table 3.1: Functional requirements

ID	Requirement
FR1	Accept a live camera or an uploaded video for each of the four channels
FR2	Detect and track people in each frame using YOLOv8 (person class only)
FR3	Assign people to zones and compute each zone's load against its allotted capacity
FR4	Estimate the fill rate, expected count after ten minutes and time to full for each zone
FR5	Classify each zone as safe, moderate or critical, and raise alerts when the level persists
FR6	Detect fire and mark the affected zone critical and blocked
FR7	Compute congestion-aware evacuation routes and a region-wise movement sequence
FR8	Show live panels, capacity, prediction and flow tables, fire and weather panels, register and event log on a web dashboard
FR9	Store crowd events and risk alerts in a database
FR10	Continue operating when a camera is unavailable, and log the fault

3.3 SYSTEM ARCHITECTURE

The system is organized as a pipeline of four groups of modules (Figure 3.2). Video enters through the **input and pre-processing module**, which accepts CCTV streams, webcams or uploaded clips. The **AI detection and tracking module** finds and follows each person. The

density and risk engine assigns people to zones, computes load and classifies risk. The **UI and alert dashboard module** presents a live crowd map, raises alerts and hands the current zone loads to the **route optimizer**, which produces the evacuation guidance. Because the optimizer reads live loads, its output changes as the crowd moves.

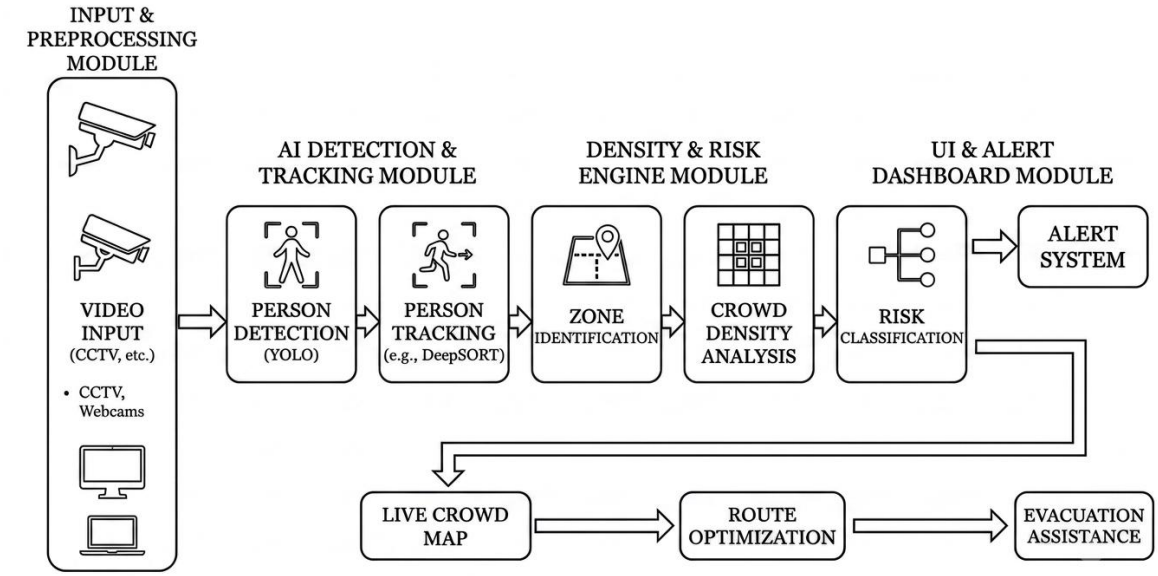


Figure 3.2: Architecture of the crowd monitoring and evacuation assistance system

Table 3.2: Modules and their responsibilities

Module	Responsibility	Technology
Input and pre-processing	Read frames from cameras or video files for each channel	OpenCV
Detection and tracking	Detect people and keep temporary identities; flow speed and direction	YOLOv8n, PyTorch, DeepSORT-style tracking
Density and risk engine	Zone load, fill rate, forecast, risk level, fire check	Python, NumPy
Route optimizer	Congestion-aware A*/Dijkstra and region-wise sequencing	Python
Dashboard and alerts	Live panels, tables, alerts, event log, register	Flask, HTML5, CSS3, JavaScript
Storage	Crowd events and risk alerts	SQLite (PostgreSQL optional)

3.4 DATA FLOW / WORKFLOW

The context diagram (Figure 3.3) shows the system exchanging data with the cameras, an optional weather source and the operators. The level-1 diagram (Figure 3.4) decomposes it into six processes and two data stores.

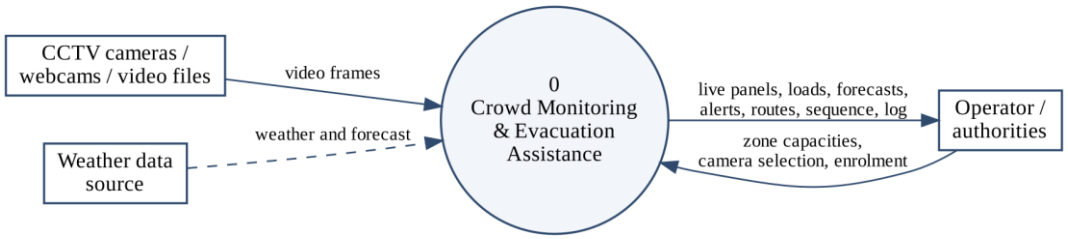


Figure 3.3: Data-flow diagram – Level 0 (context)

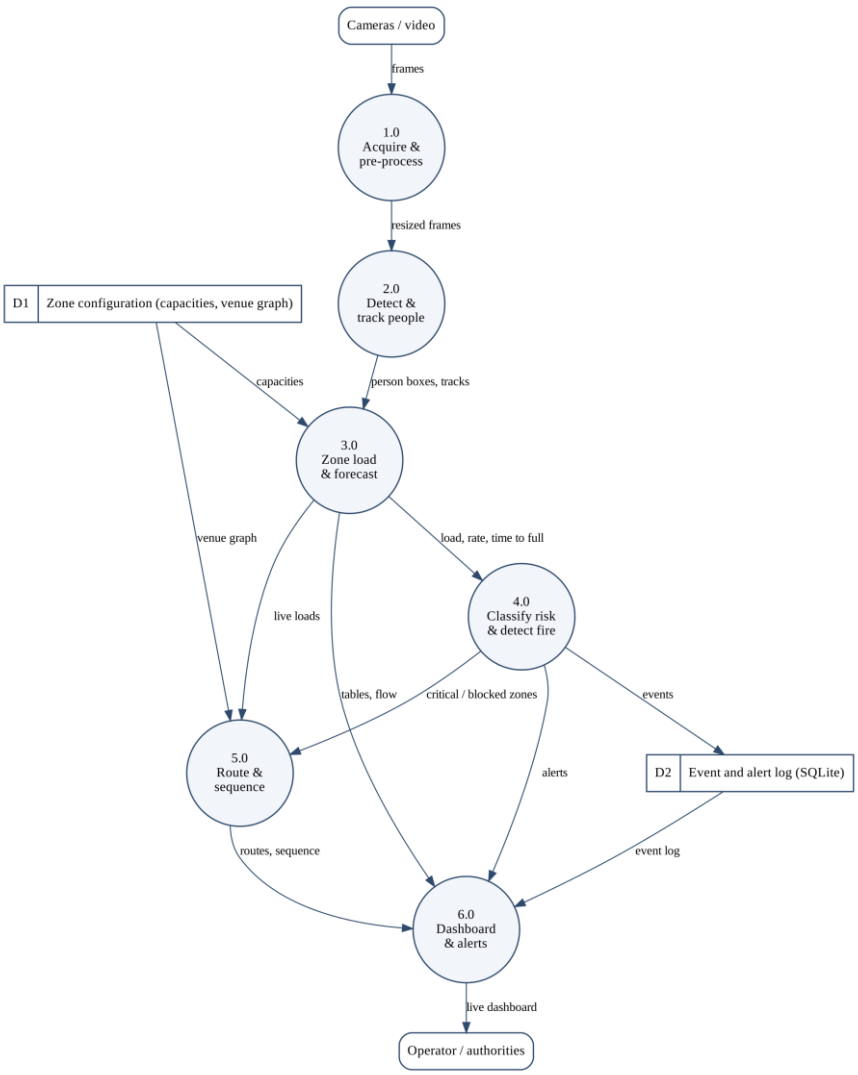


Figure 3.4: Data-flow diagram – Level 1

Frames are acquired (1.0) and passed to detection and tracking (2.0). Process 3.0 combines person tracks with the zone capacities in D1 to compute loads and forecasts. Process 4.0 classifies risk and checks for fire, writing events to the log in D2. Process 5.0 uses the live loads, the blocked zones and the venue graph in D1 to compute routes and the movement sequence. Process 6.0 presents everything on the dashboard. Figure 3.5 shows the processing loop that runs for every frame.

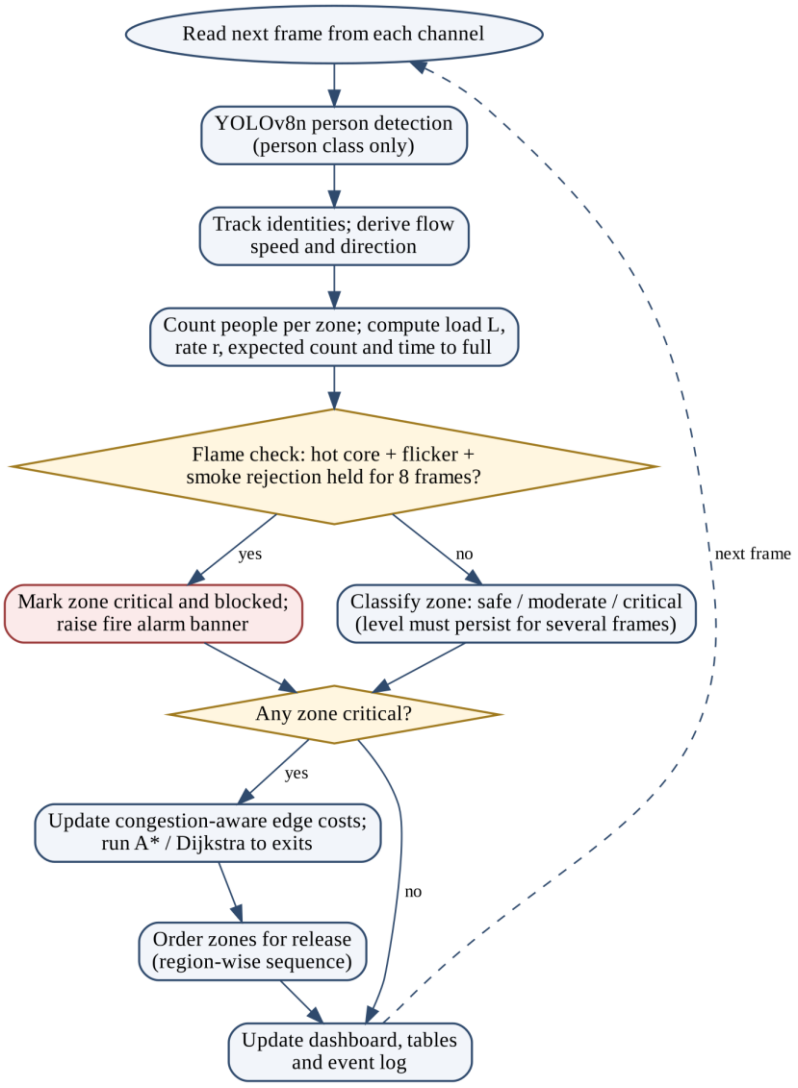


Figure 3.5: Per-frame processing workflow

3.5 HARDWARE REQUIREMENTS

Table 3.3: Hardware requirements

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
GPU	NVIDIA dedicated GPU, CUDA-enabled recommended for real-time frame rates
Memory	8 GB minimum; 16 GB recommended for AI inference
Storage	20 GB or more free SSD space
Cameras	CCTV cameras or webcams; video files may be used for testing

3.6 SOFTWARE REQUIREMENTS

Table 3.4: Software requirements

Component	Specification
Programming languages	Python (computer vision and AI inference); JavaScript ES6+, HTML5 and CSS3 (dashboard)
Computer vision and AI libraries	OpenCV [22], PyTorch [23], Ultralytics YOLOv8 [6], NumPy
Back-end framework	Flask (streams video and serves analysis results as JSON)
Database	SQLite, with PostgreSQL as the option for larger deployments
Versions	Python 3.14.6 (64-bit); OpenCV 4.13.0; PyTorch 2.13.0 (CPU build); Ultralytics 8.4.132 (YOLOv8n); Flask 3.1.3; face_recognition 1.3.0 / dlib 20.0.1; NumPy 2.5.2; Pillow 12.3.0; Microsoft Edge 156.0.4314.50; Windows 11 Home

CHAPTER 4 – IMPLEMENTATION

4.1 METHODOLOGY

The project was planned over fourteen weeks in three phases (Figure 4.1). **Planning and study** covered problem identification and objectives, the literature survey and research gap, and the system architecture and design. **Development** covered dataset collection and pre-processing, YOLO person detection and tracking, zone segmentation and density estimation, the risk classification and alert module, congestion-aware route planning (A*/Dijkstra), and the web dashboard and live crowd map. **Validation and delivery** covers system integration and testing, performance evaluation, documentation and the final demonstration, with reviews at weeks 4, 9 and 14.

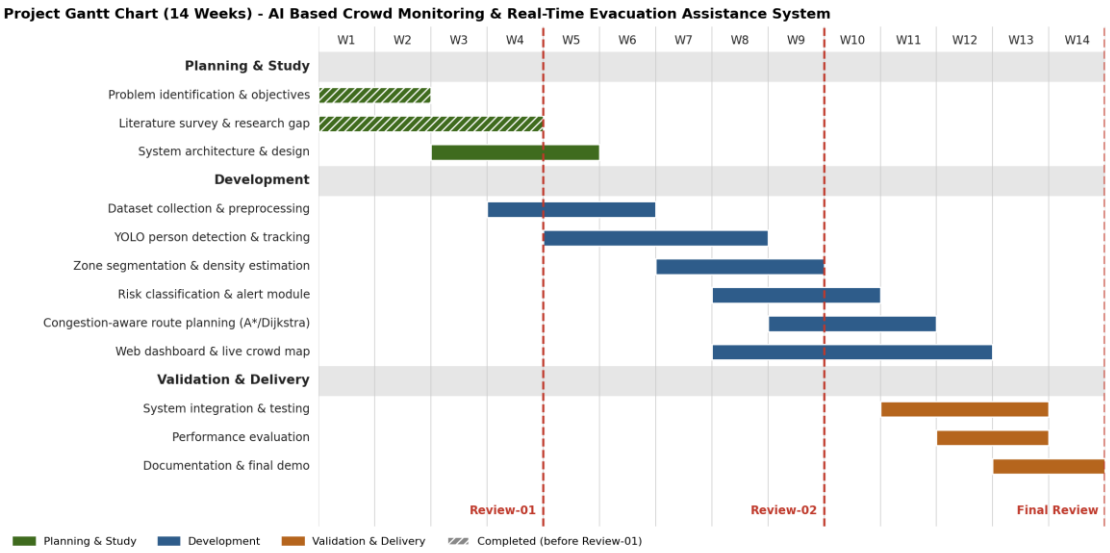


Figure 4.1: Project Gantt chart (14 weeks)

4.2 IMPLEMENTATION

4.2.1 Video Input and Pre-processing Module

The prototype console handles four channels. Each channel can be attached to a live camera or fed from an uploaded video clip, which made testing possible without access to a real venue. Frames are read with OpenCV and processed one by one. If a camera cannot be opened, the fault is written to the event log and the console continues with the remaining channels.

4.2.2 Person Detection and Tracking Module

A pretrained YOLOv8 nano model detects people in each frame, and only the person class is kept [6], [7]. The weights are loaded at start-up and the event is logged. Detections are passed to a tracker that assigns each person a temporary identity, following the DeepSORT idea of matching by both motion and appearance [8]. Tracking stops one person from being counted

twice when a detection flickers, and it gives the displacement between frames, from which the dashboard derives a flow speed in metres per second and a dominant direction for each zone.

4.2.3 Zone Load and Forecast Module

The venue is divided into named zones, each fed by its own camera channel and given an allotted capacity. In the final prototype (AEGIS-CMS) these are Z1 Main Entrance (15), Z2 Phase 1 – Main Hall (25), Z3 Phase 2 – Food Court (20) and Z4 Phase 3 – Exit Corridor (10), for a combined 70 people; the conference paper used an earlier configuration (Entrance 60, Main Hall 150, Food Court 80 and Stage Area 200). The displayed head count of each zone is the median of the last five frames, and the state of each zone is expressed as a load:

$$L(i, t) = N(i, t) / C(i) \times 100\% \tag{4.1}$$

where N(i, t) is the number of people counted in zone i and C(i) is its allotted capacity. A ratio makes zones of different sizes comparable and fits the way pedestrian planning guidance grades walkways by the space each person has [17]. Every 5 s the count of each zone is saved to SQLite, and the forecast fits a least-squares line c = m·t + b to the n samples (t, c) of the last 120 s, with each sum taken over those samples. The slope m (people per second), the count projected T = 300 s (five minutes) beyond the latest sample t_n, and the time left before the zone is full are:

$$m = (n\Sigma tc - \Sigma t\Sigma c) / (n\Sigma t^2 - (\Sigma t)^2), \quad b = (\Sigma c - m\Sigma t) / n \tag{4.2}$$

$$\hat{N}(i, t + T) = \max\{ 0, m \cdot (t_n + T) + b \} \tag{4.3}$$

$$T_{full} = (C(i) - N(i, t)) / m, \quad \text{for } m > 0 \tag{4.4}$$

The dashboard shows the trend in people per minute (60·m), the count expected five minutes ahead and the time to full. A linear forecast was chosen deliberately: it is easy to explain to an operator, needs no training data and runs in real time on a CPU. **Illustrative example:** if the Main Hall (capacity 25) holds 12 people and has gained 3 people per minute (m = 0.05 people/s), then L = 48%, the expected count after five minutes is about 12 + 0.05 × 300 = 27, and the hall will be full in (25 − 12) / 0.05 = 260 s, about 4 min 20 s. These numbers illustrate the equations and are not measurements.

4.2.4 Risk Classification and Alert Module

Each zone is labeled safe, moderate or critical from its current load (Table 4.1): safe below 75% of capacity, moderate from 75% up to just under 100%, and critical at 100% or more. Because the displayed count is already the median of the last five frames, a single bad frame cannot raise an alarm, so an overcrowding alert is issued as soon as a zone enters the moderate

(WARNING) or critical (CRITICAL) level, with no further persistence rule. A fire must be detected in six consecutive frames (about 0.75 s at the processing rate of 8 frames per second), and a VIP sighting needs two face matches within 3 s. Alerts appear on the dashboard and are written to the event log with the zone name and a timestamp.

Table 4.1: Risk levels and system response

Level	Condition	System response
Safe	Load below 75% of capacity	Monitor and log
Moderate	Load from 75% to just under 100%, or a bottleneck predicted within 5 min	Highlight zone; WARNING alert; show predicted bottleneck
Critical	Load of 100% or more, or a fire is confirmed	CRITICAL alert; alarm banner; downstream zones flagged

4.2.5 Fire Detection Module

Fire is the most obvious trigger for an evacuation, so each channel also runs a lightweight flame check. It keeps bright, saturated orange-to-red pixels, rejects regions that fill their bounding box like a flat wall (extent above 0.72), requires frame-to-frame flicker in the region's area (relative spread of at least 0.12), and declares a fire only after the signature holds over six consecutive frames. A confirmed fire raises a CRITICAL alert, lights the alarm banner in the dashboard header and is listed in the threat monitor.

4.2.6 Predictive Bottleneck Solver

In the final build, the route-optimizer block of Figure 3.2 is realized as a predictive bottleneck solver, which forecasts crowd build-up in each zone up to five minutes ahead so that staff can redirect people before a zone becomes overcrowded. The venue is modelled as a directed graph whose nodes are the four zones and whose edges are the directions in which people normally move (Figure 4.2): Z1 leads to Z2, Z2 to Z3 and Z4, and Z3 to Z4, the final zone. Using the forecast of Section 4.2.3, a zone is flagged as a likely bottleneck when:

$$\hat{N}(i, t+T) \geq 0.75 \cdot C(i) \quad and \quad m > 0 \tag{4.5}$$

that is, when the zone is projected to reach the warning level within five minutes and its count is still rising. If a zone has fewer than two samples in the window, no trend is fitted, its projected count is taken as its current count, and it is not flagged. In the example of Section 4.2.3, $27 \geq 0.75 \times 25 = 18.75$ and $m > 0$, so the Main Hall is flagged with about 4 min 20 s left before it becomes critical.

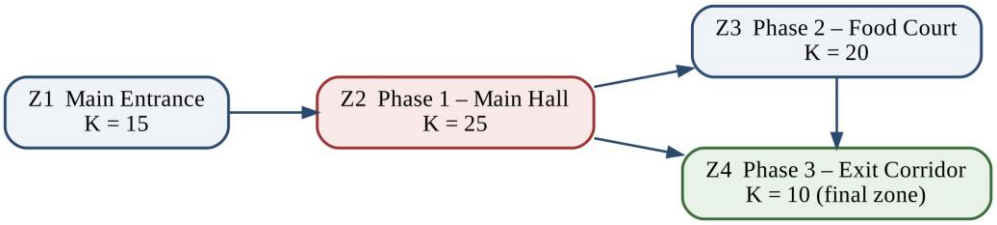


Figure 4.2: Venue zone-flow graph used by the bottleneck solver (AEGIS-CMS)

4.2.7 Downstream Risk and Bottleneck Alert Module

When a zone is flagged, every zone it leads to in the flow graph is marked as a downstream risk: if the Main Hall (Z2) is flagged, the Food Court (Z3) and the Exit Corridor (Z4) are shown as the likely next bottlenecks, and the dashboard reads "LIKELY BOTTLENECK → Z3, Z4". Staff can then open other routes or hold people back before those zones fill up. For each flagged zone a BOTTLENECK alert is logged at WARNING severity with the current count, the projected count, the trend and the estimated time to critical; to avoid repeated alerts, each zone raises at most one bottleneck alert every 60 s. The flow graph and zone capacities are set by hand in the configuration file.

4.2.8 Dashboard and Alert Module

A Flask web server streams video and serves the analysis results as JSON to a front end built with HTML5, CSS3 and JavaScript. The dashboard has a live panel for each channel with detection boxes and a per-zone load bar, a phase-capacity table, a crowd-prediction table, flow speed and direction for each zone, a fire panel, a weather panel with a short forecast and a note on its likely effect on the crowd, a register in which authorized staff can enroll a person and see which zone they are in, and a running event log. Crowd events and risk alerts are stored in SQLite.

4.3 MODULE INTEGRATION

The modules form one loop per frame (Figure 3.5): a detection on a live feed reaches the capacity table, the forecast, the alarm, the bottleneck solver and the log without manual steps. Each camera channel runs in its own thread at about 8 frames per second, so a missing camera does not stop the others, and a background loop updates zone status and predictions every 2 s. Fire detection, risk classification and the bottleneck solver write to one alert table, and the dashboard, which polls the server every 1.5 s, shows forecasts and alarms in a single threat-monitor timeline so that an operator reads them together.

CHAPTER 5 – RESULTS, CONCLUSION AND FUTURE WORK

5.1 RESULTS AND EVALUATION

The prototype was evaluated by running it end to end and checking that every part of the chain behaves as designed; it has not yet been run on a large crowd. In the test described in the conference paper, a laptop webcam was attached to the Entrance channel with a single person in view, and a short flame video was played on a phone in front of the camera to exercise the fire path. The other three channels were left unattached and correctly showed a no-signal state. Table 5.1 lists what was observed.

Table 5.1: Observed prototype behavior (from the conference paper)

Function	Observed behavior
Start-up	Four channels initialized; YOLOv8 nano loaded and logged
Detection	Person count shown per channel; boxes drawn on the live panel
Fire check	Flame boxed on Entrance; alarm banner raised; other zones clear
Capacity	1 of 60 at Entrance (2%); 1 of 490 in total
Flow	Entrance flow 0.45 m/s, north-east
Prediction	Trend, rate per minute and expected count listed for every zone
Event log	Fire and predicted-bottleneck events time-stamped in one list
Fault handling	An unavailable camera (Main Hall) was logged; other channels kept running

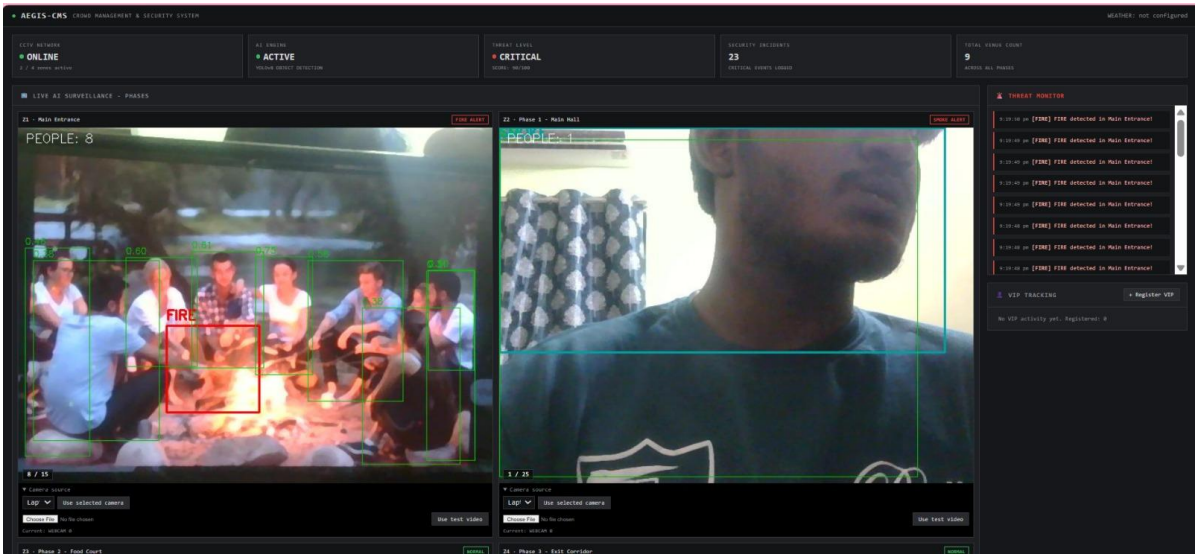


Figure 5.1: Dashboard screenshot from the project presentation

Figure 5.1, from the project presentation, shows an earlier run of the AEGIS-CMS dashboard with two of four zones active: the Z1 Main Entrance panel shows 8 people against a capacity of 15 with a fire region boxed, Z2 Main Hall shows 1 person against 25, and the threat monitor

lists time-stamped fire entries beside a VIP tracking panel. AEGIS-CMS is the final configuration of the prototype, with capacities of 15 (Z1), 25 (Z2), 20 (Z3) and 10 (Z4), a combined 70 people; the capacities in the conference paper belong to an earlier configuration. This version has no evacuation-route view, so its bottleneck-solver table is shown instead (Figure 5.2).

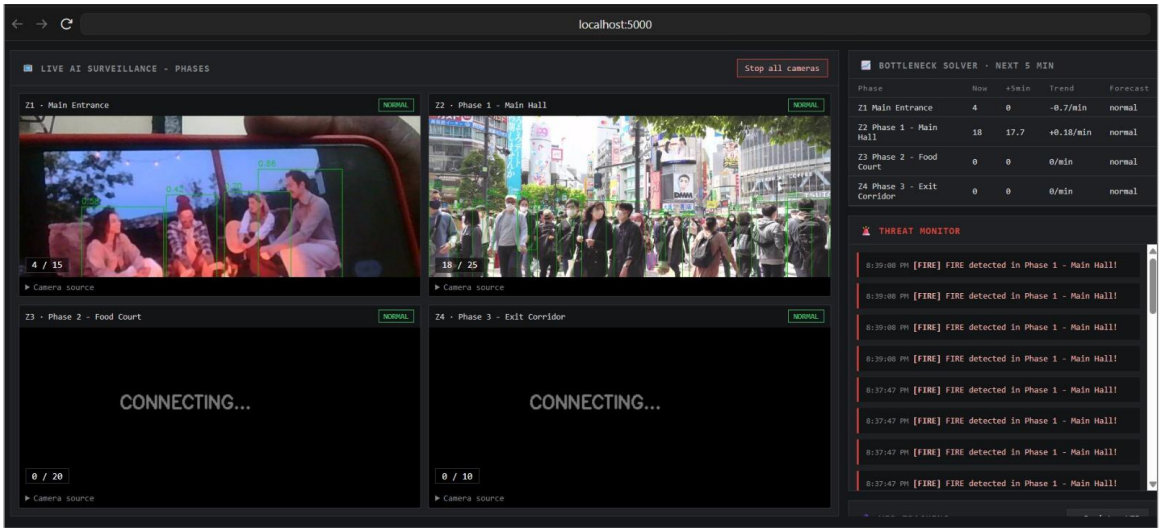


Figure 5.2: Final dashboard with the bottleneck-solver table

In the final run (Figure 5.2), test videos were attached to Z1 and Z2, while Z3 and Z4 were left unattached and showed a connecting state. Z1 counted 4 people against 15 (27%) and Z2 counted 18 against 25 (72%); both were correctly labeled NORMAL because their loads were below 75%. The solver table listed the current count, the five-minute forecast and the trend for every zone (Z1: 4, 0, −0.7/min; Z2: 18, 17.7, +0.18/min), and no zone was flagged because none was both rising and projected to reach 75% of capacity. Figure 5.3(a) shows the Main Hall video with 19 people detected, and Figure 5.3(b) shows alerts from an earlier run in which counts were rising quickly.



Figure 5.3: (a) YOLOv8n detections on the Main Hall video; (b) threat-monitor alerts

Table 5.2: Test cases

ID	Function	Input / action	Expected result	Status
TC01	Start-up	Launch console	Four channels and YOLOv8n loaded; logged	Observed
TC02	Detection	One person in front of webcam	Count 1; box drawn	Observed
TC03	Fire	Flame video in front of camera	Fire confirmed after 6 frames; CRITICAL alert; alarm banner	Observed
TC04	Fault handling	Unavailable camera	Fault logged; other channels continue	Observed
TC05	Capacity	Test videos on Z1 and Z2	Correct count, load and level	Pass
TC06	Forecast	Zone filling quickly	Correct trend, 5-min forecast and time to critical	Pass
TC07	Risk	Load raised above capacity	Zone turns critical at once; CRITICAL alert logged	Not verified
TC08	Bottleneck	Zone forecast to reach 75% and rising	BOTTLENECK alert with forecast and ETA	Pass
TC09	Video input	Test videos on two channels	Both processed independently; others show connecting	Pass
TC10	False alarm	Street video with red and orange signs	No fire declared	Fail
TC11	Accuracy	Dense street video	Counting error against ground truth	Not measured
TC12	Performance	Four live channels	Frames per second and alert latency reported	Not measured

Measured results. In the run of Figure 5.3(b), the solver flagged the Main Hall at 17 people with a trend of +11.99 people per minute and a projected 79.0 people in five minutes; its estimated time to critical, about 40 s, agrees with Eq. (4.4): $(25 - 17) / (11.99 / 60) \approx 40$ s. The Main Entrance was flagged at 16 people (+12.1/min), already above its capacity of 15, so its time to critical was shown as "soon". The log in Figure 5.2 also contains FIRE alerts for the Main Hall while its feed was a street scene with bright red and orange signboards and no flame, a false alarm (TC10). By design, each channel is processed at about 8 frames per second, so a fire is confirmed about 0.75 s after it first appears, and the dashboard refreshes every 1.5 s. Counting error against an annotated dataset and the frame rate with four live channels have not yet been measured.

5.2 DISCUSSION

The central idea of the project is simple: an exit that looks closest on a map is not the best choice if it is already jammed, and a system that knows the live crowd can say so. The prototype

demonstrates the full chain from camera to early warning on one dashboard, which distinguishes it from counting-only systems [9], [10], from anomaly detectors that raise alarms without plans [13], and from routing methods that rely on simulation or user devices [18], [19]. The linear forecast and rule-based fire check keep the system explainable and lightweight, at the cost of slower reaction to sudden surges and possible false fire alarms. Because the system observes people, a real deployment must also address privacy, especially for the enrolment register, which stores personal information.

5.3 CHALLENGES AND LIMITATIONS

- The demonstration used one person and one active camera; behavior with hundreds of people, heavy occlusion or poor lighting has not been measured.
- Detection-based counting with a small YOLO model loses accuracy in very dense scenes, where density-map networks such as CSRNet are usually stronger [3].
- Flow speed in metres per second depends on calibrating pixels to real distances for each camera, which needs a site survey.
- The zone-flow graph and zone capacities are set by hand, and the linear forecast is slow to react to sudden surges.
- The flame check is heuristic and raised false fire alarms on a street video with bright red and orange signboards; the enrolment register raises privacy questions.
- The final build forecasts bottlenecks and flags downstream zones, but it does not yet compute evacuation routes or a region-wise release order.

5.4 CONCLUSION

This project presents a prototype that connects what a camera sees to what an evacuation planner recommends. It counts people per zone, grades each zone by load, forecasts when it will fill, checks for fire, and forecasts bottlenecks up to five minutes ahead while flagging the downstream zones at risk, all on one dashboard. By moving crowd management from passive CCTV monitoring to proactive, automated decision support, the system aims to improve public safety, reduce response time and lower the risk of crowd-related disasters in large public spaces.

5.5 FUTURE WORK

- **Measurement:** counting error against public crowd datasets, frames per second per channel, and the time from a critical event to an alert.
- **Dense scenes:** add a density-map model and compare it with detection-based counting.

- **Evacuation routing:** add congestion-aware routes (A*/Dijkstra) and a region-wise release order on the zone-flow graph, and validate them against an agent-based simulation in the spirit of social-force models [1].
- **Deployment:** move inference to edge hardware near the cameras, and add push alerts and digital signage so that guidance reaches the crowd and not just the control room.
- **Calibration:** per-camera pixel-to-metre calibration and automatic tuning of zone thresholds.

REFERENCES

[1] D. Helbing, I. Farkas, and T. Vicsek, "Simulating dynamical features of escape panic," Nature, vol. 407, no. 6803, pp. 487–490, 2000.

[2] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-image crowd counting via multi-column convolutional neural network," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Las Vegas, NV, USA, 2016, pp. 589–597.

[3] Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes," in Proc. IEEE CVPR, Salt Lake City, UT, USA, 2018, pp. 1091–1100.

[4] V. A. Sindagi and V. M. Patel, "A survey of recent advances in CNN-based single image crowd counting and density estimation," Pattern Recognit. Lett., vol. 107, pp. 3–16, 2018.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, Las Vegas, NV, USA, 2016, pp. 779–788.

[6] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[7] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Proc. Eur. Conf. Comput. Vis. (ECCV), 2014, pp. 740–755.

[8] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in Proc. IEEE Int. Conf. Image Process. (ICIP), Beijing, China, 2017, pp. 3645–3649.

[9] M. H. K. Khel, K. A. Kadir, S. Khan, M. N. M. Noor, H. Nasir, N. Waqas, and A. Khan, "Realtime crowd monitoring—Estimating count, speed and direction of people using hybridized YOLOv4," IEEE Access, vol. 11, pp. 56368–56379, 2023, doi: 10.1109/ACCESS.2023.3272481.

[10] M. Rezaei and M. Azarmi, "DeepSOCIAL: Social distancing monitoring and infection risk assessment in COVID-19 pandemic," Applied Sciences, vol. 10, no. 21, Art. no. 7514, 2020, doi: 10.3390/app10217514.

[11] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, "Social LSTM: Human trajectory prediction in crowded spaces," in Proc. IEEE CVPR, Las Vegas, NV, USA, 2016, pp. 961–971.

[12] V. Mahadevan, W. Li, V. Bhalodia, and N. Vasconcelos, "Anomaly detection in crowded scenes," in Proc. IEEE CVPR, San Francisco, CA, USA, 2010, pp. 1975–1981.

[13] C. Direkoğlu, "Abnormal crowd behavior detection using motion information images and convolutional neural networks," IEEE Access, vol. 8, pp. 80408–80416, 2020, doi: 10.1109/ACCESS.2020.2990355.

[14] G. Farnebäck, "Two-frame motion estimation based on polynomial expansion," in Proc. Scandinavian Conf. Image Anal. (SCIA), 2003, pp. 363–370.

[15] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, 1959.

[16] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Trans. Syst. Sci. Cybern., vol. 4, no. 2, pp. 100–107, 1968.

[17] J. J. Fruin, Pedestrian Planning and Design. New York, NY, USA: Metropolitan Association of Urban Designers and Environmental Planners, 1971.

[18] Z. Zhang, H. Liu, Z. Jiao, Y. Zhu, and S.-C. Zhu, "Congestion-aware evacuation routing using augmented reality devices," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2020.

[19] S. Huang, J. Ji, Y. Wang, W. Li, and Y. Zheng, "A machine vision-based method for crowd density estimation and evacuation simulation," Safety Science, vol. 167, Art. no. 106285, 2023, doi: 10.1016/j.ssci.2023.106285.

[20] R. Roy Choudhury, A. Karmakar, and R. P. Mitra, "Drishti AI-Event Guardian: An intelligent real-time crowd monitoring and emergency response system for mass gathering events," arXiv preprint arXiv:2606.05185, 2026.

[21] D. Perkins and S. Chakraborty, "GPEvac: GNN-based PPO for adaptive evacuation routing during shooting events," arXiv preprint arXiv:2609.16163, 2026.

[22] G. Bradski, "The OpenCV library," Dr. Dobb's J. Softw. Tools, 2000.

[23] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Adv. Neural Inf. Process. Syst. 32 (NeurIPS), 2019, pp. 8024–8035.

APPENDIX – SOURCE CODE

This appendix lists the main source code of the AEGIS-CMS (Crowd Monitoring System). The listings are shortened versions of the real project files: error handling, logging, drawing code and comments have been trimmed so that each listing shows only the main logic. The names, thresholds and formulas are the same as in the full code.

Listing 1: config.py – zones, capacities, thresholds

Listing 2: core/detection_engine.py – YOLOv8 person detection / head count

Listing 3: core/zone_manager.py – overcrowding status per zone

Listing 4: core/fire_smoke_detector.py – fire detection

Listing 5: core/vip_manager.py – VIP face registration and recognition

Listing 6: core/bottleneck_predictor.py – 5-minute bottleneck prediction

Listing 7: core/stream_manager.py – per-camera processing loop

Listing 8: app.py – Flask web server and API

Listing 1: config.py - venue zones and thresholds

```
ZONES = {
    "Z1": {"name": "Main Entrance",           "capacity": 15},
    "Z2": {"name": "Phase 1 - Main Hall",     "capacity": 25},
    "Z3": {"name": "Phase 2 - Food Court",    "capacity": 20},
    "Z4": {"name": "Phase 3 - Exit Corridor", "capacity": 10},
}

# Which zone people normally move into next
ZONE FLOW GRAPH = {"Z1": ["Z2"], "Z2": ["Z3", "Z4"], "Z3": ["Z4"], "Z4": []}

WARNING_THRESHOLD = 0.75        # 75% of capacity
CRITICAL THRESHOLD = 1.0        # 100% of capacity

DETECTION_CONFIDENCE = 0.25     # YOLO person confidence
COUNT SMOOTHING WINDOW = 5      # median of last 5 frames
TARGET_PROCESS_FPS = 8

FIRE CONSECUTIVE FRAMES = 6     # frames in a row before a fire alert
FIRE_FLICKER_MIN_STD_RATIO = 0.12
FIRE MAX EXTENT = 0.72

PREDICTION_HORIZON_SEC = 300    # predict 5 minutes ahead
HISTORY WINDOW SEC = 120        # use the last 2 minutes of counts
SAMPLE INTERVAL SEC = 5         # save a head count every 5 seconds

FACE MATCH TOLERANCE = 0.45
VIP_CONFIRMATION_HITS = 2       # 2 matches ...
VIP_CONFIRMATION_WINDOW_SEC = 3 # ... within 3 seconds
```

Listing 2: core/detection_engine.py - person detection and head count

```
from collections import deque
from ultralytics import YOLO

class PersonDetector:
    def __init__(self):
        self.model = YOLO("yolov8n.pt")
        self.recent counts = deque(maxlen=COUNT SMOOTHING WINDOW)

    def detect(self, frame):
        result = self.model.predict(frame, conf=DETECTION CONFIDENCE,
                                    iou=0.55, imgsz=768, classes=[0])[0]  # class 0 =
person
        boxes = [tuple(map(int, b.xyxy[0])) for b in result.boxes]

        # Smooth the count: median of the last 5 frames
        self.recent_counts.append(len(boxes))
        head_count = sorted(self.recent_counts)[len(self.recent_counts) // 2]
        return head_count, boxes
```

Listing 3: core/zone_manager.py - overcrowding status

```
class ZoneState:
    def __init__(self, zone_id, name, capacity):
        self.zone id, self.name, self.capacity = zone id, name, capacity
        self.head count = 0
        self.status = "NORMAL"

    def update count(self, count):
        self.head_count = count
        load = count / self.capacity
        if load >= CRITICAL THRESHOLD:
            self.status = "OVERCROWDED"
        elif load >= WARNING THRESHOLD:
            self.status = "WARNING"
        else:
            self.status = "NORMAL"
        return self.status

class ZoneManager:
    def update(self, zone_id, count):
        zone = self.zones[zone id]
        old status = zone.status
        new_status = zone.update_count(count)
        database.add zone snapshot(zone id, count)        # every 5 s, used by the
predictor

        if new status in ("WARNING", "OVERCROWDED") and new status != old status:
            severity = "CRITICAL" if new status == "OVERCROWDED" else "WARNING"
            database.add_alert("OVERCROWD", zone_id, severity,
                               f"Overcrowding {new status} in {zone.name}: "
                               f"head count {count}/{zone.capacity}")
```

Listing 4: core/fire_smoke_detector.py - fire detection

```
import cv2, numpy as np
from collections import deque

class FireDetector:
    def __init__(self):
        self.fire streak = 0
        self.area history = deque(maxlen=12)

    def detect(self, frame):
        # 1. Colour: keep bright, saturated orange/yellow/red pixels
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        mask = cv2.inRange(hsv, np.array([0, 120, 170]), np.array([30, 255, 255]))

        # 2. Largest region: its area and how much of its box it fills
        area, extent = largest contour area and extent(mask)
        self.area_history.append(area if area >= FIRE_MIN_AREA else 0)

        # 3. Flicker: real flames change size from frame to frame
        flicker = np.std(self.area_history) / max(np.mean(self.area_history), 1)

        is candidate = (area >= FIRE MIN AREA
                        and extent <= FIRE_MAX_EXTENT              # irregular shape,
not a wall
                        and flicker >= FIRE FLICKER MIN STD RATIO) # flickering, not
static

        # 4. Persistence: must be seen in 6 frames in a row
        self.fire_streak = self.fire_streak + 1 if is_candidate else 0
        return self.fire_streak >= FIRE_CONSECUTIVE_FRAMES
```

Listing 5: core/vip_manager.py - VIP registration and recognition

```
import face recognition
from PIL import Image, ImageOps

class VIPManager:
    def register_vip(self, name, tier, photo_path):
        img = ImageOps.exif transpose(Image.open(photo path)).convert("RGB")
        rgb = np.array(img)

        faces = face recognition.face locations(rgb)
        if not faces:
            raise ValueError("No face detected in the photo.")
        encodings = face recognition.face encodings(rgb, faces)
        encoding = encodings[largest face index(faces)]   # use the largest face

        vip id = database.next vip id()                   # e.g. VIP-0001
        database.add_vip(vip_id, name, tier, photo_path, encoding)
        return vip_id

    def identify_faces(self, frame):
        rgb = cv2.cvtColor(frame, cv2.COLOR BGR2RGB)
        locations = face recognition.face locations(rgb)
        results = []
        for box, encoding in zip(locations, face recognition.face encodings(rgb,
locations)):
            distances = face_recognition.face_distance(self.known_encodings,
encoding)
            best = distances.argmin()
            if distances[best] <= FACE_MATCH_TOLERANCE:
                results.append({"vip id": self.known ids[best], "box": box})
        return results
```

Listing 6: core/bottleneck_predictor.py - Predictive Bottleneck Solver

```
def linear trend(points):
    """Least-squares line c = m*t + b through [(t, count), ...]."""
    n = len(points)
    sum t  = sum(t for t, c in points)
    sum_c  = sum(c for t, c in points)
    sum tc = sum(t * c for t, c in points)
    sum tt = sum(t * t for t, c in points)
    m = (n * sum_tc - sum_t * sum_c) / (n * sum_tt - sum_t ** 2)
    b = (sum c - m * sum t) / n
    return m, b                                    # m = people per second

def predict zone(zone id, current count):
    capacity = ZONES[zone id]["capacity"]
    points = database.get_recent_snapshots(zone_id, last_seconds=HISTORY_WINDOW_SEC)
    if len(points) < 2:
        return {"predicted_count": current_count, "will_bottleneck": False}

    m, b = linear trend(points)
    t_now = points[-1][0]
    predicted = max(0, m * (t now + PREDICTION HORIZON SEC) + b)

    will_bottleneck = predicted >= WARNING_THRESHOLD * capacity and m > 0

    eta = None
    if m > 0 and current_count < capacity:
        eta = (CRITICAL THRESHOLD * capacity - current count) / m   # seconds to full

    return {"rate_per_min": m * 60, "predicted_count": predicted,
            "will bottleneck": will bottleneck, "eta seconds": eta}

def predict_all(zone_counts):
    predictions = {z: predict zone(z, c) for z, c in zone counts.items()}
    for z, p in predictions.items():
        # Zones fed by a bottleneck zone are flagged as the next risk
        p["downstream risk"] = ZONE FLOW GRAPH[z] if p["will bottleneck"] else []
        if p["will_bottleneck"]:
            database.add_alert("BOTTLENECK", z, "WARNING", "Predicted
bottleneck ...")
    return predictions
```

Listing 7: core/stream_manager.py - per-camera processing loop

```
class ZoneWorker(threading.Thread):           # one thread per camera / zone
    def run(self):
        cap = cv2.VideoCapture(self.source)    # webcam index or test-video file
        while not self.stop event.is set():
            ok, frame = cap.read()
            if not ok:
                cap.set(cv2.CAP PROP POS FRAMES, 0)   # loop the test video
                continue

            head count, boxes = self.detector.detect(frame)
            fire = self.fire_detector.detect(frame)

            if self.frame idx % 15 == 0:              # face recognition is slower
                for vip in self.vip_manager.identify_faces(frame):
                    if self.confirmed(vip):           # 2 matches within 3 s
                        record_vip_location(vip["vip_id"], self.zone_id)

            if fire:
                database.add_alert("FIRE", self.zone_id, "CRITICAL",
                                   f"FIRE detected in
{ZONES[self.zone id]['name']}!")

            self.state.head count = head count
            self.state.jpeg = cv2.imencode(".jpg", draw boxes(frame,
boxes))[1].tobytes()
            time.sleep(1 / TARGET_PROCESS_FPS)        # about 8 frames per second
```

Listing 8: app.py - Flask web server and API

```
from flask import Flask, Response, render template, jsonify, request

app = Flask(  name  )
stream manager = StreamManager()
predictor = BottleneckPredictor()

def background loop():                          # runs every 2 seconds
    while True:
        stream manager.tick zone counts()     # update zone status + save snapshots
        latest["predictions"] = predictor.predict all(zone counts())
        time.sleep(2)

@app.route("/")
def index():
    return render template("index.html", zones=ZONES)

@app.route("/video_feed/<zone_id>")           # live MJPEG stream for one camera
def video feed(zone id):
    def frames():
        while True:
            yield (b"--frame\r\nContent-Type: image/jpeg\r\n\r\n"
                   + stream_manager.get_jpeg(zone_id) + b"\r\n")
            time.sleep(1 / 12)
    return Response(frames(), mimetype="multipart/x-mixed-replace; boundary=frame")

@app.route("/api/overview")                   # polled by the dashboard every 1.5 s
def api overview():
    return jsonify({"zones": zone_status_list(),
                    "predictions": latest["predictions"],
                    "alerts": get recent alerts(limit=30)})

@app.route("/api/vip/register", methods=["POST"])
def api vip register():
    vip_id = stream_manager.vip_manager.register_vip(
        request.form["name"], request.form["tier"],
save_upload(request.files["photo"]))
    return jsonify({"ok": True, "vip_id": vip_id})

@app.route("/api/zone/<zone_id>/source/video", methods=["POST"])
def api_set_video(zone_id):                   # use an uploaded test video as the
camera
    path = save_upload(request.files["video"])
    stream_manager.set_zone_source(zone_id, path)
    return jsonify({"ok": True})

if __name__ == "__main__":
    threading.Thread(target=background loop, daemon=True).start()
    app.run(host="0.0.0.0", port=5000, threaded=True)
```